

Javascript Basic for React

1. Cấu trúc và cú pháp cơ bản

a. Khai báo biến: có thể dùng các từ khóa let, const hoặc var để khai báo biến trong Javascript

b. Kiểu dữ liệu: string, number, object <Map>, array, undefined, null, symbol, bigint

- với undefined: có nghĩa là chưa được gán giá trị nào cả ví dụ như: khai báo biến không có giá trị, truy cập một phần tử không có trong object, một hàm không có return

- với null: có nghĩa là giá trị của nó là null biến đó không có giá trị.

c. Toán tử số học Arithmetic trong Javascript

`5 + 3 // 8`

`10 - 2 // 8`

`2 * 4 // 8`

`16 / 2 // 8`

`17 % 3 // 2 (phần dư)`

`2 ** 3 // 8 (2^3)`

`'5' + 3 // "53"` (nối chuỗi vì có string) khi có string thì phép toán cộng coi là nối chuỗi

`'5' - 3 // 2` (ép kiểu sang number) khi thực hiện phép - thì phải ép kiểu sang number để thực hiện phép toán

Javascript sẽ cố ép kiểu string sang number trước khi thực hiện các phép toán trừ phép cộng vì nó hiểu là nối chuỗi.

Khi không thể ép kiểu số được thì khi thực hiện phép toán nó sẽ trả ra "NaN" <Not a Number>.

Ép kiểu ở đây kể cả với null, false, true chúng sẽ có giá trị lần lượt là 0,0,1. Null và False ép kiểu số sẽ là 0 còn True ép sẽ thành 1.

Còn với Undefined thì không thể bị ép kiểu về number

với || or && thì hơi khác chút a || b: a OR b: nếu a là truethy thì trả về a ngược lại trả về b trả về giá trị đầu tiên nếu true

a && b : nếu a là falsethy thì trả về a ngược lại trả về b; trả về giá trị đầu tiên nếu false và giá trị 2 nếu cả 2 cùng là true và cuối cùng thì || chỉ cần 1 giá trị truethy để dừng còn && thì cần tất cả là truethy để đi đến cuối

Vậy truethy và falsethy là gì? : đơn giản thì truethy là có gì đó tồn tại, không trống rỗng, còn Falsy thì là Không tồn tại hoặc rỗng

**** Hàm mũi tên Arrow Function**

truyền thông function abx () {}

cách hiện đại hơn : const abc = () => {

đặc biệt nếu mà viết sau mũi tên dấu ngoặc tròn thì không cần dùng từ khóa return ngược lại nếu viết dấu ngoặc nhọn thì cần thêm từ khóa return để trả về kết quả.

**** import và export**

- Với export thì có 2 kiểu export đó là default export và named export đơn giản là nếu trong file react đó ta export một phần tử thì ta dùng default export còn nếu ta export nhiều phần thì dùng named export
 - với default export thì ta thêm từ khóa default sau từ export là dc ví dụ:
export default Page;
 - với named export thì ta phải thêm dấu ngoặc nhọn vào trong ví dụ như:
export {Page, Home}
- Với import thì tương tự như vậy nếu là default thì ta không cần phải thêm dấu ngoặc nhọn khi import còn với named export thì ta cần thêm dấu ngoặc nhọn khi ta import component nào đó.

vd: import Page from "" // default export
import {Page, Home } from "" // named export

**** đặc biệt với default export** vì trong file ta chỉ export một thứ thôi nên javascript biết ta export thứ gì nên khi import ta có thể thay đổi tên tùy ý còn với named thì không được cần dùng đúng tên giống như khi export.

**** Toán tử 3 ngôi trong javascript cũng giống như trong flutter:** `đk ? kq1 : kq2 ;`

Ngoài ra ta cũng có thể sử dụng toán tử `&&`: `isLogin && <p> </p>`: nếu isLogin là truthy thì render thẻ `<p>` còn falsy thì không render gì hết.
Trong react thì nếu là falsy thì không hiển thị gì. cách hoạt động của toán tử `&&` vẫn giống hệt như javascript a truthy tiếp đến b truthy hiển thị còn không thì thôi. Nhưng mà trong JSX thì các thẻ html nó là một object nên nó luôn là truthy.

Ta cũng có thể sử dụng toán tử `||` : cái này thì hiển thị khi mà đk trước sai tương tự như `&&` thôi falsy -> JSX truthy -> trả về JSX
với `??` : thì nếu về trước là null hoặc undefined thì trả về giá trị JSX đằng sau.

**** Object** được hiểu giống như kiểu Map trong flutter và dart được khai báo với cặp key-value

- Khi làm việc với object thì ta cần quan tâm đến 2 kỹ thuật đó là destructuring và spread <bóc tách và gộp sao chép>
với object destructuring thì ta có thể tạo các biến nhanh khi bóc tách các giá trị từ object nhanh chóng tiện lợi

vd: `const {age, class} = cat;` và lưu ý sâu hơn: Tên biến cần phải giống như tên key bên trong object để có thể lấy được giá trị của nó.

— Lưu ý 2: Khi khai báo một object nếu mà ta thấy có key và value giống hệt nhau thì chỉ cần key là đc không cần value, tương tự như nếu mang giá trị boolean thì chỉ cần key thôi nó tự hiểu là true còn nếu là false thì bắt buộc phải khai báo đầy đủ key: value

**** Kỹ thuật spread :** sao chép và chỉnh sửa object mảng cực kỳ nhanh và tiện

ví dụ như `cat1 { "a": 12, "b": 13 }`

thì ta có thể tạo object khác như: `const cat2 = {...cat1, "b": 1}`: dùng 3 dấu chấm cho object, array để tạo mảng khác giống mảng kia, sau dấu phẩy sẽ là thêm hoặc chỉnh sửa các trường object. Ở đây nó sẽ thực hiện thêm đối với array hoặc sửa nếu object với key trùng nhau hoặc thêm nếu key không tồn tại trong object cũ.

***** Thao tác với mảng map và filter và khi thêm code javascript bên trong JSX thì cần đặt code trong {JS code}**

- map dùng để vẽ một mảng các phần tử giao diện
- filter dùng để lọc phần tử theo điều kiện

******* khi làm việc với các đoạn mã bất đồng bộ và nó không trả về kết quả ngay thì nó không trả về kết quả thông thường thì nó sẽ trả về một Promise và khi hoàn tất thì kết quả cũng sẽ được nhét vào trong promise cho mình sử dụng

– Promise thì nó đại diện cho kết quả có trong tương lai. Nó có 3 trạng thái pending đang chờ xử lý, fullfill đã hoàn thành và reject là lỗi. Xử lý Promise cũng giống như khi sử dụng Future với Flutter vậy. dùng then để xử lý kết quả khi thành công còn dùng catch để xử lý lỗi exception

— Lưu ý: Nhưng mà với Javascript thì khi lấy kết quả từ Promise ra nó không biết là json hay gì thì cần chuyển nó về Json để có thể sử dụng được nó.

- Cũng có thể xử dụng async và await giống như trong flutter để xử lý kết quả bất đồng bộ.

```
vd: const fetchData = async () => {  
    const response = await fetchData();  
    const data = await response.json();  
    // xử lý response ở đây nhé  
}
```

— Lưu ý 2: code bất đồng bộ bắt buộc phải dùng try catch để xử lý vì bất đồng

bộ rất dễ xảy ra lỗi trời ơi đất hỡi.

Và khi gọi nhiều api hay code bất đồng bộ xong xong với nhau thì đừng await từng cái một thế kia sẽ mất rất nhiều thời gian xử lý từng cái thay vào đó ta có thể sử dụng Promise.allSettled ({}).